

MODULE 02/09

# TypeScript

JavaScript avec des super-pouvoirs

■ Durée : 1 semaine

■ Projet : Social Seller

■ Niveau : Débutant →  
Opérationnel

---

## Objectifs de la semaine

- ✓ Comprendre pourquoi TypeScript existe et ce qu'il apporte par rapport à JavaScript
  - ✓ Maîtriser les types primitifs, les interfaces et les types unions
  - ✓ Utiliser les generics pour écrire du code réutilisable
  - ✓ Typer correctement async/await et les appels d'API
  - ✓ Lire et comprendre le code TypeScript de Social Seller
- 

## Planning de la semaine

| JOUR 1                                                                                                                                                                                                                                                                                | Types de base             | 2-3h |
|---------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|---------------------------|------|
| <ul style="list-style-type: none"><li>• Pourquoi TypeScript ? Erreurs que JS ne détecte pas, TS si</li><li>• Types primitifs : string, number, boolean, null, undefined</li><li>• Tableaux typés : string[], number[], Array</li><li>• Types union : string   number   null</li></ul> |                           |      |
| JOUR 2                                                                                                                                                                                                                                                                                | Interfaces et objets      | 2-3h |
| <ul style="list-style-type: none"><li>• interface vs type — quand utiliser quoi</li><li>• Propriétés optionnelles (?), readonly, index signatures</li><li>• Étendre une interface : interface B extends A</li><li>• Typer les fonctions : paramètres et valeur de retour</li></ul>    |                           |      |
| JOUR 3                                                                                                                                                                                                                                                                                | Generics et utilitaires   | 2-3h |
| <ul style="list-style-type: none"><li>• Generics : function identity(arg: T): T</li><li>• Generics constraints : T extends object</li><li>• Types utilitaires : Partial, Required, Pick, Omit</li><li>• Record, Readonly</li></ul>                                                    |                           |      |
| JOUR 4                                                                                                                                                                                                                                                                                | Async, erreurs et modules | 2-3h |
| <ul style="list-style-type: none"><li>• Promise et async/await typé</li><li>• Gestion des erreurs : try/catch avec unknown</li><li>• Modules : import/export, default export vs named export</li><li>• tsconfig.json — les options importantes</li></ul>                              |                           |      |
| JOUR 5                                                                                                                                                                                                                                                                                | Mini-projet de la semaine | 3-4h |
| <ul style="list-style-type: none"><li>• Créer une mini-API de gestion de contacts en TypeScript pur</li><li>• Définir les interfaces : Contact, CreateContactInput, UpdateContactInput</li></ul>                                                                                      |                           |      |

- Implémenter un CRUD typé en mémoire (tableau de contacts)
- Ajouter une fonction `search(items: T[], query: string): T[]`
- Compiler avec `tsc --noEmit` : zéro erreur

# Concepts clés détaillés

## Les types de base

```
// Types primitifs
const nom: string = 'Mohamed';
const age: number = 28;
const actif: boolean = true;

// Tableaux
const tags: string[] = ['mobile', 'saas'];
const scores: Array<number> = [95, 87, 100];

// Union – plusieurs types possibles
let id: string | number;
id = 'abc-123'; // OK
id = 42; // OK

// Nullable
let prenom: string | null = null;
prenom = 'Fatima'; // OK
```

## Interfaces — la brique fondamentale

```
interface Tenant {
  id: string;
  name: string;
  plan: 'free' | 'pro' | 'enterprise';
  createdAt: Date;
  deletedAt?: Date; // optionnel
}

interface CreateTenantInput {
  name: string;
  plan?: 'free' | 'pro'; // optionnel à la création
}
```

```
// Étendre une interface
interface AdminTenant extends Tenant {
  adminEmail: string;
}

// Typer une fonction
function createTenant(input: CreateTenantInput): Tenant {
  return {
    id: crypto.randomUUID(),
    name: input.name,
    plan: input.plan ?? 'free',
    createdAt: new Date(),
  };
}
```

## Generics — code réutilisable

```
// Sans generic : dupliqué pour chaque type
function firstString(arr: string[]): string | undefined {
  return arr[0];
}

// Avec generic : une seule fonction pour tous les types
function first<T>(arr: T[]): T | undefined {
  return arr[0];
}

first(['a', 'b', 'c']); // TypeScript infère T = string
first([1, 2, 3]); // TypeScript infère T = number

// Exemple réel de Social Seller
async function graphGet<T>(path: string, token: string): Promise<T> {
  const res = await fetch(`https://graph.facebook.com${path}`);
  return res.json() as T;
}

const data = await graphGet<{ data: User[] }>('/me/accounts', token);
```

## Async/await typé

```
// Promise<T> – ce que la fonction retourne en asynchrone
async function fetchUser(id: string): Promise<User | null> {
```

```

try {
  const res = await fetch(`/api/users/${id}`);
  if (!res.ok) return null;
  return res.json() as Promise<User>;
} catch (err: unknown) {
  // err est 'unknown', pas 'any' — plus sûr
  if (err instanceof Error) {
    console.error(err.message);
  }
  return null;
}

// Types utilitaires les plus utiles
type PartialUser = Partial<User>; // tous les champs optionnels
type UserPreview = Pick<User, 'id' | 'name'>; // seulement 2 champs
type UserWithoutId = Omit<User, 'id'>; // tout sauf id

```

■ `tsc --noEmit` est ta commande de vérification. Elle compile sans écrire de fichiers — seulement pour détecter les erreurs de type. Sur Social Seller, on la lance après chaque prompt Claude Code.

## Exercices pratiques

### Exercice 1 — Typer un objet existant (20 min)

Prends ce code JavaScript et ajoute les types TypeScript :

```

// À typer en TypeScript
function formatOrder(order) {
  return {
    label: `Commande #${order.id.slice(0, 8)}`,
    amount: `${order.totalAmount} FCFA`,
    status: order.status,
  };
}

```

### Exercice 2 — Interface et validation (30 min)

- Définir une interface `Message` avec : `id`, `content`, `direction` (`'inbound' | 'outbound'`), `createdAt`, `isRead`
- Créer une fonction `markAsRead(message: Message): Message` qui retourne une copie avec `isRead=true`

- Tenter de passer `isRead` comme `string` — observer l'erreur TypeScript

### Exercice 3 — Generic utile (40 min)

- Écrire une fonction `paginate(items: T[], page: number, size: number): { data: T[], total: number, page: number }`
  - Tester avec un tableau de `strings` puis un tableau d'objets
  - Ajouter une contrainte : `T extends { id: string }`
- 

## Ressources pour aller plus loin

- Documentation officielle : [typescriptlang.org/docs](https://typescriptlang.org/docs)
- Playground interactif : [typescriptlang.org/play](https://typescriptlang.org/play)
- Total TypeScript (Matt Pocock) : [totaltypescript.com](https://totaltypescript.com)
- Type Challenges : [github.com/type-challenges/type-challenges](https://github.com/type-challenges/type-challenges)